

MECHATRONICS AND IOT

ASSIGNMENT REPORT – 9

TITLE:

Design and develop a Wireless Environmental Parameter Monitoring System for Smart Industries. The system remotely monitors environmental conditions such as temperature, humidity, and air quality in industrial environments.

TEAM MEMBERS:

HARISANKARAN S

(24MER013)

MUTHUKUMAR M

(24MER034)

MONESHWARAN C

(24MER032)

PROJECT OVERVIEW

Industries require continuous monitoring of environmental conditions to maintain a safe and efficient working environment. Parameters such as temperature, humidity, and air quality can affect employee safety, equipment performance, product quality, and energy consumption. The proposed Wireless Environmental Parameter Monitoring System uses an ESP32 microcontroller, DHT22 sensor, MQ-135 air-quality sensor, Wi-Fi, MQTT communication, and a cloud platform to remotely monitor industrial environmental conditions. The collected sensor data is transmitted wirelessly, stored in the cloud, and displayed for real-time monitoring and historical analysis.

PROBLEM STATEMENT

- Excessive temperature in industrial environments.
- High or low humidity levels.
- Poor air quality due to gases, smoke, or pollutants.
- Lack of continuous environmental monitoring.
- Delayed detection of abnormal environmental conditions.
- Manual monitoring of industrial parameters.
- Lack of remote access to environmental data.
- Potential risks to workers, equipment, and production processes.
- Increased energy consumption due to improper environmental control.

PROPOSED SOLUTION

The proposed system uses an ESP32 controller, DHT22 sensor, and MQ-135 air-quality sensor to continuously monitor industrial environmental conditions. The ESP32 collects temperature, humidity, and air-quality readings and compares them with predefined threshold values. The monitored data is transmitted through Wi-Fi using MQTT communication to a cloud platform.

The system provides:

- Real-time environmental monitoring.
- Wireless data transmission.
- Remote access through a web or mobile dashboard.
- Cloud-based data storage.
- Historical data analysis.
- Automatic warning for abnormal conditions.
- Local alarm indication during critical conditions.

WORKING PRINCIPLE

The DHT22 sensor measures temperature and humidity, while the MQ-135 sensor monitors air quality. The ESP32 continuously reads data from these sensors and processes the readings according to predefined threshold values.

The environmental condition is classified as:

- NORMAL
- WARNING
- CRITICAL

The data is displayed locally and transmitted through Wi-Fi using MQTT to the cloud platform.

SYSTEM ARCHITECTURE

Sensors → ESP32 → Wi-Fi → MQTT → Cloud Platform → Remote Dashboard

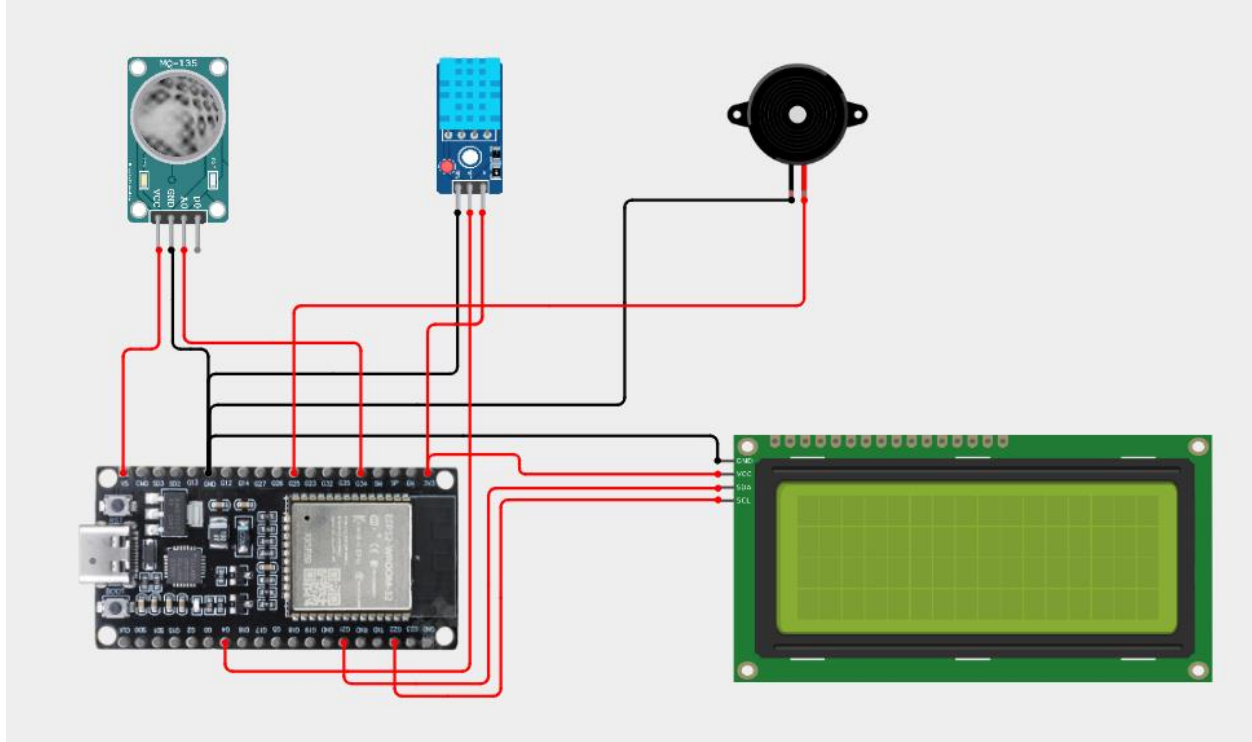
INVENTORY THRESHOLD VALUES

Parameter	Normal	Warning	Critical
Temperature	22–25 °C	25–28 °C	>28 °C
Humidity	40–60 %	60–70 %	>70 %
Air Quality	<100	100–150	>150

PARAMETERS MONITORED

Parameter	Sensor/Device	Purpose
Temperature	DHT22	Monitor industrial temperature
Humidity	DHT22	Monitor environmental humidity
Air Quality	MQ-135	Detect air-quality conditions
System Status	ESP32	Monitor system operation
Warning Status	Buzzer/LED	Indicate abnormal conditions
Cloud Status	Wi-Fi/MQTT	Monitor data transmission

CIRCUIT DESIGN



ALGORITHM

- **START**
- Initialize the ESP32 microcontroller.
- Initialize the DHT22 sensor.
- Initialize the MQ-135 sensor.
- Initialize the LCD/OLED display.
- Initialize the buzzer and LED indicators.
- Connect the ESP32 to the Wi-Fi network.
- Connect to the MQTT broker.
- Read temperature from the DHT22 sensor.
- Read humidity from the DHT22 sensor.
- Read the air-quality value from the MQ-135 sensor.
- Compare all readings with predefined threshold values.
- Determine the environmental condition as NORMAL, WARNING, or CRITICAL.
- Display the sensor readings locally.
- Publish the data using MQTT.
- Upload and store the data on the cloud platform.
- Activate the buzzer during critical conditions.
- Repeat the process continuously.
- **STOP**

CODING

```
#include "AdafruitIO_WiFi.h"

#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#include <DHT.h>

// =====
// WIFI DETAILS
// =====

#define WIFI_SSID    "OnePlus Nord CE5 7C5C"
#define WIFI_PASSWORD "qkts5737"

// =====
// ADAFRUIT IO DETAILS
// =====

#define IO_USERNAME  "Hair_13"
#define IO_KEY       "aio_EMRC40vci7DQ5Amz5kvOKh6WKHI0"

// Create Adafruit IO WiFi object
AdafruitIO_WiFi io(
  IO_USERNAME,
  IO_KEY,
  WIFI_SSID,
  WIFI_PASSWORD
);

// =====
// FEEDS
// =====

AdafruitIO_Feed *temperatureFeed =
  io.feed("temperature");

AdafruitIO_Feed *humidityFeed =
  io.feed("humidity");
```

```
AdafruitIO_Feed *mq135Feed =  
  io.feed("mq135");
```

```
AdafruitIO_Feed *airQualityAlertFeed =  
  io.feed("air-quality-alert");
```

```
// =====  
// DHT11  
// =====
```

```
#define DHT_PIN 4  
#define DHT_TYPE DHT11
```

```
DHT dht(DHT_PIN, DHT_TYPE);
```

```
// =====  
// MQ135 AIR QUALITY SENSOR  
// =====
```

```
#define MQ135_PIN 34
```

```
// =====  
// BUZZER  
// =====
```

```
#define BUZZER_PIN 25
```

```
// =====  
// OLED  
// =====
```

```
#define SCREEN_WIDTH 128  
#define SCREEN_HEIGHT 64
```

```
Adafruit_SSD1306 display(  
  SCREEN_WIDTH,  
  SCREEN_HEIGHT,  
  &Wire,  
  -1  
);
```

```

// =====
// VARIABLES
// =====

float temperature = 0;
float humidity = 0;

int mq135Value = 0;

bool airQualityAlert = false;

// =====
// TIMING
// =====

unsigned long lastSend = 0;

const unsigned long sendInterval =
    15000;

// =====
// SETUP
// =====

void setup()
{
    Serial.begin(9600);

    delay(1000);

    // -----
    // DHT
    // -----

    dht.begin();

    // -----
    // SENSOR PINS
    // -----

    pinMode(MQ135_PIN, INPUT);

```

```
pinMode(BUZZER_PIN, OUTPUT);
```

```
digitalWrite(  
  BUZZER_PIN,  
  LOW  
);
```

```
// -----  
// OLED  
// -----
```

```
Wire.begin(21, 22);
```

```
if (!display.begin(  
  SSD1306_SWITCHCAPVCC,  
  0x3C))  
{
```

```
  Serial.println(  
    "OLED not detected!"  
  );
```

```
}  
else  
{
```

```
  display.clearDisplay();
```

```
  display.setTextColor(  
    SSD1306_WHITE  
  );
```

```
  display.setTextSize(1);
```

```
  display.setCursor(10, 10);
```

```
  display.println(  
    "SMART MONITOR"  
  );
```

```
  display.setCursor(10, 30);
```

```
  display.println(  
    "Starting..."
```

```
);

display.display();
}

// -----
// CONNECT TO ADAFRUIT IO
// -----

Serial.println();
Serial.println(
  "Connecting to Adafruit IO..."
);

io.connect();

// Wait for connection
while (io.status() < AIO_CONNECTED)
{

  Serial.print(".");

  delay(500);
}

Serial.println();

Serial.println(
  "Connected to Adafruit IO!"
);

display.clearDisplay();

display.setTextSize(1);

display.setCursor(0, 10);

display.println(
  "Adafruit IO"
);

display.setCursor(0, 30);
```

```
display.println(
  "Connected!"
);

display.display();

delay(2000);
}

// =====
// LOOP
// =====

void loop()
{

  // Keep Adafruit IO connection alive
  io.run();

  // -----
  // SEND SENSOR DATA
  // -----

  if (
    millis() - lastSend >=
    sendInterval
  )
  {

    lastSend = millis();

    // =====
    // DHT11
    // =====

    temperature =
      dht.readTemperature();

    humidity =
      dht.readHumidity();
```

```
if (isnan(temperature) ||
    isnan(humidity))
{

    Serial.println(
        "DHT11 reading failed!"
    );

}
else
{

    Serial.print(
        "Temperature: "
    );

    Serial.print(
        temperature
    );

    Serial.println(" C");

    Serial.print(
        "Humidity: "
    );

    Serial.print(
        humidity
    );

    Serial.println(" %");
}


// =====
// MQ135
// =====

mq135Value =
    analogRead(MQ135_PIN);

Serial.print(
    "MQ135: "
```

```
);

Serial.println(
  mq135Value
);

// =====
// AIR QUALITY ALERT
// =====

// Adjust this threshold after testing

if (mq135Value > 2000)
{

  airQualityAlert = true;

  digitalWrite(
    BUZZER_PIN,
    HIGH
  );

}
else
{

  airQualityAlert = false;

  digitalWrite(
    BUZZER_PIN,
    LOW
  );
}

// =====
// SEND TO ADAFRUIT IO
// =====

Serial.println();

Serial.println(
  "Sending data to Adafruit IO..."
);
```

```
if (!isnan(temperature))
{

    temperatureFeed->save(
        temperature
    );
}
```

```
if (!isnan(humidity))
{

    humidityFeed->save(
        humidity
    );
}
```

```
mq135Feed->save(
    mq135Value
);
```

```
airQualityAlertFeed->save(
    airQualityAlert ? 1 : 0
);
```

```
Serial.println(
    "Data sent successfully!"
);
```

```
// =====
// OLED
// =====
```

```
display.clearDisplay();
```

```
display.setTextColor(
    SSD1306_WHITE
);
```

```
display.setTextSize(1);
```

```
display.setCursor(0, 0);

display.print("T:");

display.print(
  temperature,
  1
);

display.print("C ");

display.print("H:");

display.print(
  humidity,
  0
);

display.println("%");

display.setCursor(0, 15);

display.print("MQ135: ");

display.println(
  mq135Value
);

display.setCursor(0, 30);

if (airQualityAlert)
{

  display.println(
    "AIR QUALITY ALERT!"
  );

}
else
{
```

```

        display.println(
            "Air Quality Normal"
        );
    }

    display.setCursor(0, 45);

    display.print("WiFi: ");

    display.println(
        "Connected"
    );

    display.display();

    Serial.println(
        "=====
    );
}
}

```

SYSTEM WORKING

The Wireless Environmental Parameter Monitoring System continuously monitors industrial environmental conditions using multiple sensors. The DHT22 sensor measures temperature and humidity, while the MQ-135 sensor monitors air quality. The ESP32 collects these readings and compares them with predefined threshold values. The current readings are displayed on an LCD or OLED display. The ESP32 then sends the monitored data wirelessly through Wi-Fi using MQTT communication to the cloud platform. The cloud platform stores the sensor readings and provides graphical visualization for real-time and historical analysis. When abnormal environmental conditions are detected, the system generates a warning. During critical conditions, a buzzer and LED can be activated. The monitoring process operates continuously to provide remote environmental supervision.

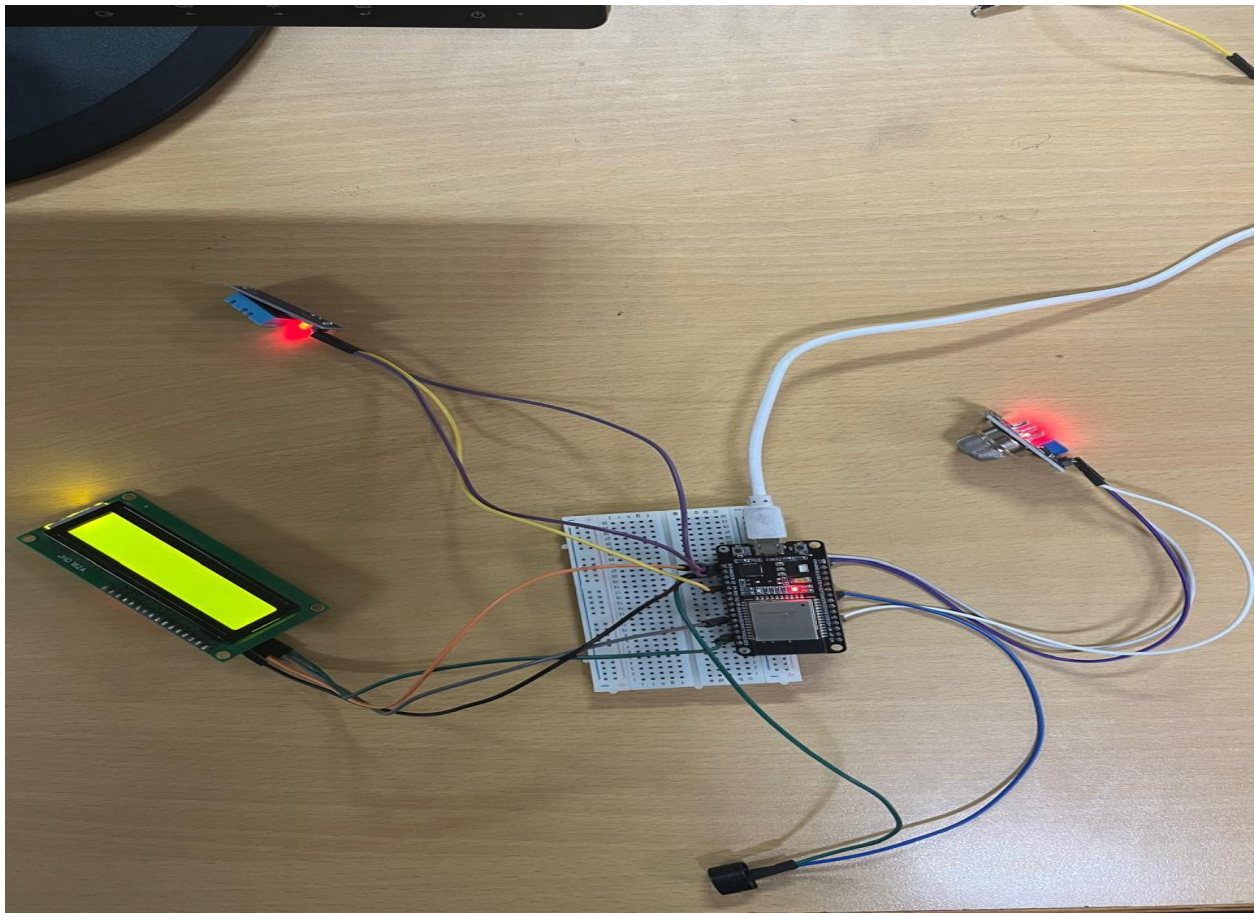
COMPONENTS

S.No.	Component	Quantity	Purpose
1	ESP32 Development Board	1	Main controller
2	DHT22 Sensor	1	Temperature and humidity monitoring
3	MQ-135 Sensor	1	Air-quality monitoring

S.No.	Component	Quantity	Purpose
4	Wi-Fi Network	1	Wireless communication
5	LCD/OLED Display	1	Display environmental parameters
6	Buzzer	1	Critical-condition alarm
7	LEDs	1 set	Status indication
8	Breadboard	1	Circuit implementation
9	Jumper Wires	1 set	Electrical connections
10	Power Supply	1	Power source

PROTOTYPE IMPLEMENTATION

The prototype is designed around an **ESP32 microcontroller** and environmental sensors. The DHT22 sensor is connected to the ESP32 to measure temperature and humidity, while the MQ-135 sensor is used to monitor air-quality conditions. The ESP32 processes the sensor data and displays the current environmental conditions locally. Using its built-in Wi-Fi capability, the ESP32 transmits the data to the cloud through MQTT communication. The cloud platform stores the monitored data and provides graphical visualization. A buzzer and LEDs are used to provide immediate alerts when the environmental parameters exceed predefined limits. The prototype therefore provides both **local monitoring and remote wireless monitoring**.



RESULTS & PERFORMANCE ANALYSIS

The developed system provides real-time monitoring of important industrial environmental parameters. The sensor readings are continuously compared with predefined threshold values to identify the environmental condition.

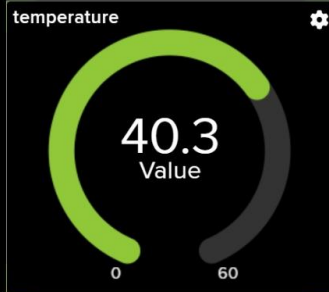
The system can:

- Continuous environmental monitoring.
- Wireless transmission of sensor data.
- Remote access to environmental information.
- Detection of abnormal conditions.
- Cloud storage of historical data.
- Graphical visualization of environmental trends.
- Immediate alerts during critical conditions. Support better construction-site planning.

Parameter	Normal	Warning	Critical
Temperature	22–25 °C	25–28 °C	>28 °C
Humidity	40–60 %	60–70 %	>70 %
Air Quality	<100	100–150	>150

PERFORMANCE

- Provides real-time monitoring of temperature, humidity, and air quality.
- Uses wireless Wi-Fi communication.
- Supports MQTT-based data transmission.
- Enables remote industrial environmental monitoring.
- Automatically detects abnormal environmental conditions.
- Provides warning and critical-condition alerts.
- Stores historical sensor data in the cloud.
- Provides graphical visualization of environmental parameters.
- Reduces the need for manual environmental monitoring.
- Helps improve industrial safety and working conditions.
- Supports efficient environmental and energy management.
- Provides a low-cost IoT solution suitable for smart industrial applications.



air-quality-alert

